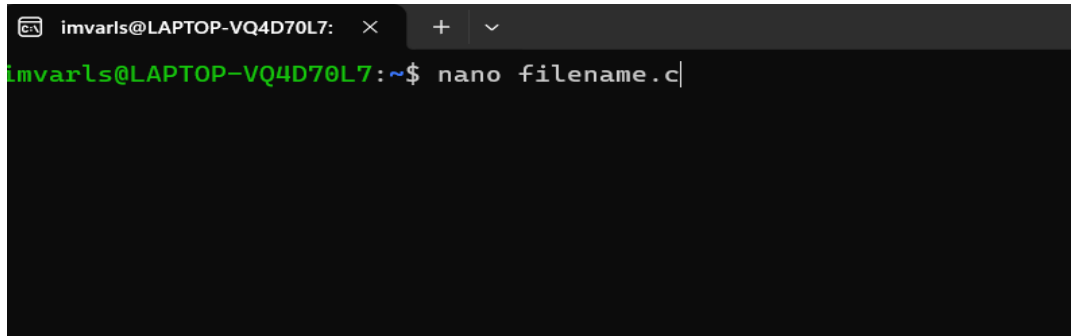


WEEK 3

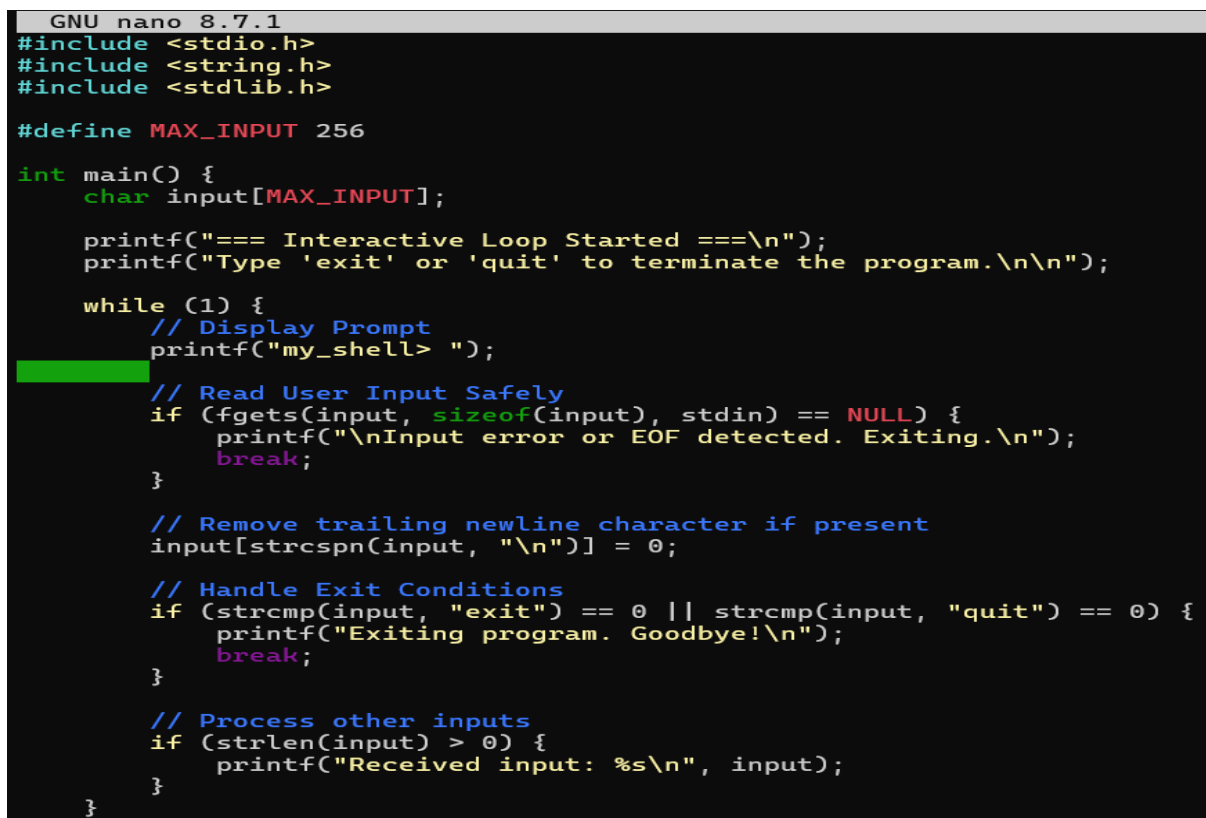
Task 1: To Create Main Loop, Display Prompt, Read User Input, Handle Exit Conditions, Design Control Flow Diagram, Test Interactive Loop

1. Create file



```
imvarls@LAPTOP-VQ4D70L7: ~$ nano filename.c
```

2. Program



```
GNU nano 8.7.1
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

#define MAX_INPUT 256

int main() {
    char input[MAX_INPUT];

    printf("=== Interactive Loop Started ===\n");
    printf("Type 'exit' or 'quit' to terminate the program.\n\n");

    while (1) {
        // Display Prompt
        printf("my_shell> ");

        // Read User Input Safely
        if (fgets(input, sizeof(input), stdin) == NULL) {
            printf("\nInput error or EOF detected. Exiting.\n");
            break;
        }

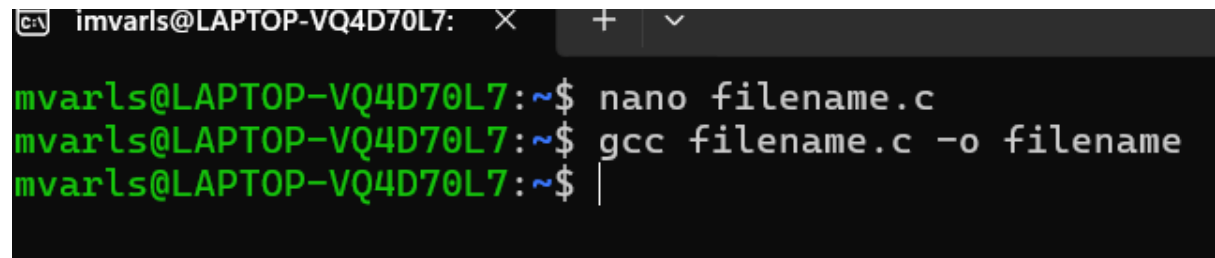
        // Remove trailing newline character if present
        input[strcspn(input, "\n")] = 0;

        // Handle Exit Conditions
        if (strcmp(input, "exit") == 0 || strcmp(input, "quit") == 0) {
            printf("Exiting program. Goodbye!\n");
            break;
        }

        // Process other inputs
        if (strlen(input) > 0) {
            printf("Received input: %s\n", input);
        }
    }
}
```

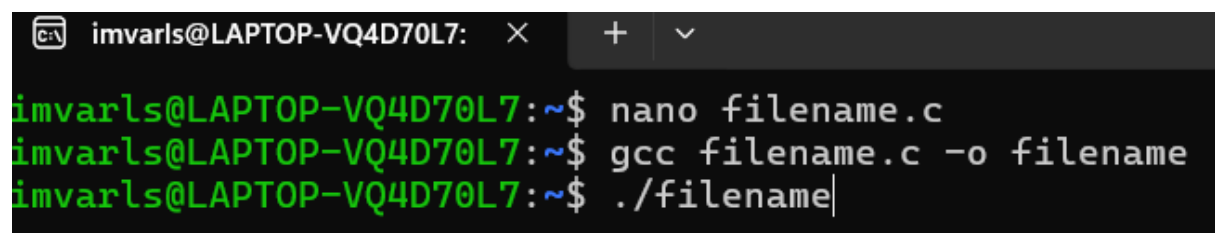
3 Then for compilation write a command

`gcc filename.c -o filename`

A terminal window with a dark background and green text. The prompt is 'imvarls@LAPTOP-VQ4D70L7: ~\$'. The user enters 'nano filename.c', then 'gcc filename.c -o filename', and finally a blank line.

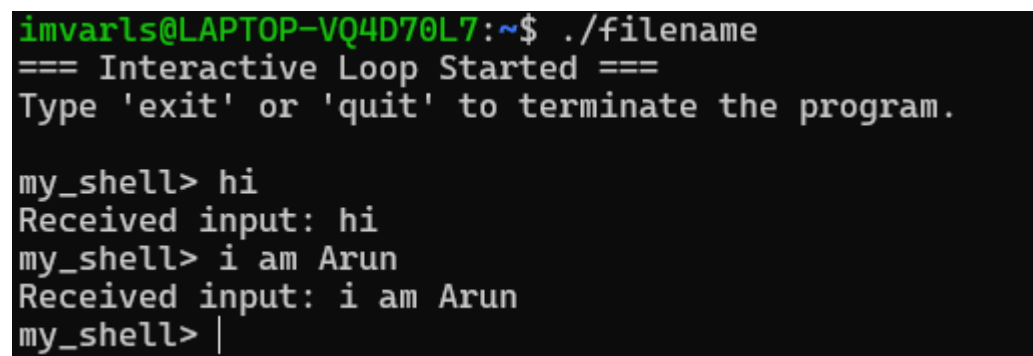
```
imvarls@LAPTOP-VQ4D70L7: ~$ nano filename.c
imvarls@LAPTOP-VQ4D70L7: ~$ gcc filename.c -o filename
imvarls@LAPTOP-VQ4D70L7: ~$
```

4. Then Enter `./filename`

A terminal window with a dark background and green text. The prompt is 'imvarls@LAPTOP-VQ4D70L7: ~\$'. The user enters 'nano filename.c', then 'gcc filename.c -o filename', and finally './filename'.

```
imvarls@LAPTOP-VQ4D70L7: ~$ nano filename.c
imvarls@LAPTOP-VQ4D70L7: ~$ gcc filename.c -o filename
imvarls@LAPTOP-VQ4D70L7: ~$ ./filename
```

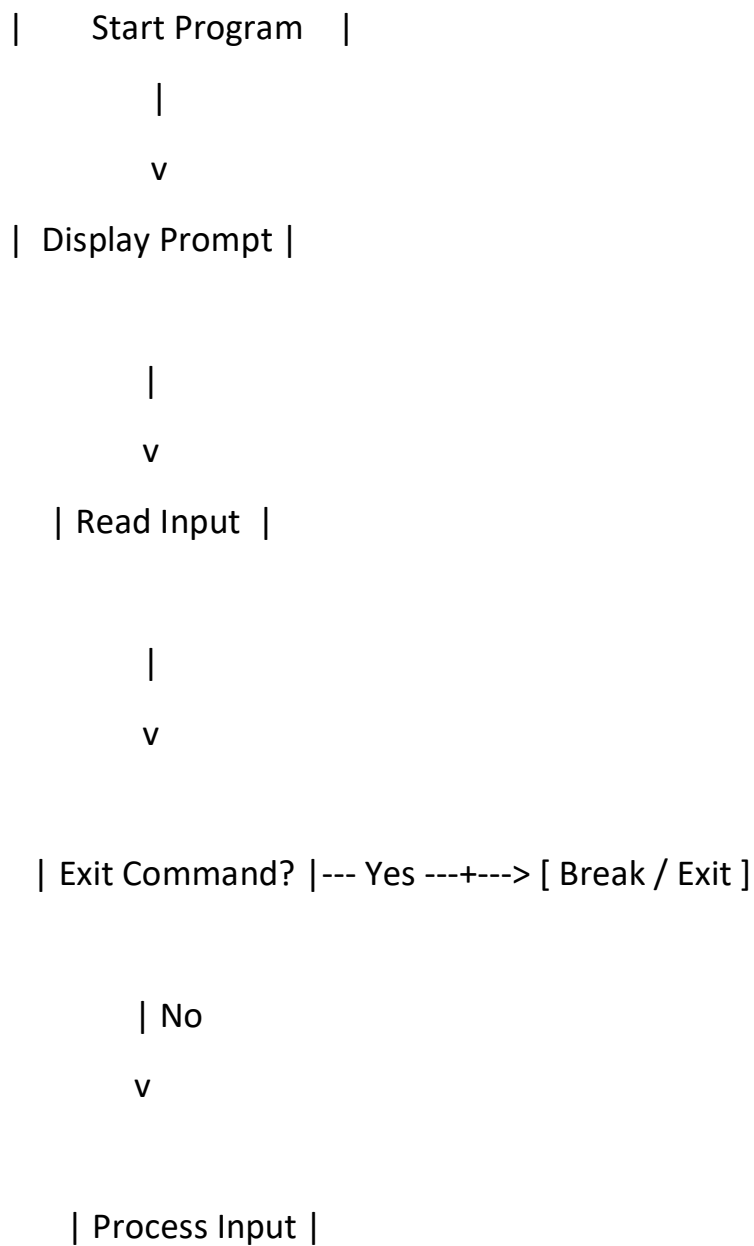
5. Output

A terminal window with a dark background and green text. The prompt is 'imvarls@LAPTOP-VQ4D70L7: ~\$'. The user enters './filename'. The output shows '=== Interactive Loop Started ===' and 'Type \'exit\' or \'quit\' to terminate the program.' followed by a prompt 'my_shell>'. The user enters 'hi', and the output is 'Received input: hi'. The user enters 'i am Arun', and the output is 'Received input: i am Arun'. The prompt 'my_shell>' is followed by a vertical bar indicating the cursor.

```
imvarls@LAPTOP-VQ4D70L7: ~$ ./filename
=== Interactive Loop Started ===
Type 'exit' or 'quit' to terminate the program.

my_shell> hi
Received input: hi
my_shell> i am Arun
Received input: i am Arun
my_shell> |
```

Control Flow



Task 2: Using the following system calls, copy the content from File1.txt to File2.txt.

1. **Open()**
2. **Read()**
3. **Write()**
4. **Close()**

open()

syntax: open(" filename.txt", modes, permissions)

Example: open("First_Prgm.txt", O_CREAT | O_WRONLY, 0644);

Table 1: Modes of Open system call

Flag	Purpose
O_RDONLY	Open for read only
O_WRONLY	Open for write only
O_RDWR	Open for both read and write
O_CREAT	Create file if it does not exist
O_TRUNC	Truncate (empty) an existing file
O_APPEND	Append data to the end of the file
O_EXCL	Fail if file already exists (used with O_CREAT)

read()

Syntax: read(int fd, void *buf, size_t count);

Example: read(fd, buffer, 100)

```
#include <stdio.h>
```

```
#include <unistd.h>
```

```
int main()
```

```
{
```

```
    char buffer[100];
```

```
    int n;
```

```
    printf("Enter some text: ");
```

```
    n = read(0, buffer, sizeof(buffer) - 1);
```

```
buffer[n] = '\0'; // Null-terminate the string
printf("You entered: %s", buffer);
return 0;
}
```

write()

Syntax: write(int fd, const void *buf, size_t count);

Example: write(fd, message, 11);

#include <unistd.h>

```
int main()
{
    write(1, "Hello World\n", 12);
    return 0;
}
```

```
imvarls@LAPTOP-VQ4D70L7:~$ nano File1.tx|
```

```
GNU nano 8.7.1 File1.tx
Operating System lab
Using Open(), Read(), Write(), and Close() system calls
```

```
imvarls@LAPTOP-VQ4D70L7:~$ nano File1.tx
imvarls@LAPTOP-VQ4D70L7:~$ nano copy.c|
```

```

GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h> // For open() flags
#include <unistd.h> // For write() and close()
#include <string.h> // For strlen()

int main() {
    int fd;
    const char *text = "Hello! This is test content created inside File1.txt.\n";

    // Open File1.txt for writing (creates it if it doesn't exist, truncates if it does)
    fd = open("File1.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd < 0) {
        perror("Error creating File1.txt");
        exit(EXIT_FAILURE);
    }

    // Write text into File1.txt
    if (write(fd, text, strlen(text)) < 0) {
        perror("Error writing to File1.txt");
        close(fd);
        exit(EXIT_FAILURE);
    }

    printf("File1.txt created and populated successfully!\n");

    // Close the file
    close(fd);

    return 0;
}

```

```

imvarls@LAPTOP-VQ4D70L7:~$ nano File1.tx
imvarls@LAPTOP-VQ4D70L7:~$ nano copy.c
imvarls@LAPTOP-VQ4D70L7:~$ gcc copy.c -o copy
imvarls@LAPTOP-VQ4D70L7:~$ ./copy
File1.txt created and populated successfully!
imvarls@LAPTOP-VQ4D70L7:~$ |

```

```

imvarls@LAPTOP-VQ4D70L7:~$ nano File1.tx
imvarls@LAPTOP-VQ4D70L7:~$ nano copy.c
imvarls@LAPTOP-VQ4D70L7:~$ gcc copy.c -o copy
imvarls@LAPTOP-VQ4D70L7:~$ ./copy
File1.txt created and populated successfully!
imvarls@LAPTOP-VQ4D70L7:~$ cat File1.tx
Operating System lab
Using Open(), Read(), Write(), and Close() system calls
imvarls@LAPTOP-VQ4D70L7:~$ |

```